WEAK-AREA REVIEW

Targeted Review

6 Topics You Missed

Databricks Certified Data Engineer Associate

- Structured Streaming trigger intervals
- ANSI compliance & spark.sql.ansi.enabled
- Setting Spark configuration properties
- Dashboard task in Lakeflow Jobs
- Converting external tables to managed (SET MANAGED)
- Unity Catalog privileges & inheritance
- 20 practice questions + one-page cheat sheet

Grounded in docs.databricks.com (retrieved Oct 2026) | © 2026 Study Guide

Contents

1	Structured Streaming Triggers	3
2	ANSI SQL Compliance	5
3	Spark Configuration Properties	7
4	Dashboard Task in Lakeflow Jobs	9
5	Convert External Tables to Managed	10
6	Unity Catalog Privileges	12
7	Practice Questions (20)	14
8	Cheat Sheet	16

How to use this guide

Each section covers one documentation page you flagged as a weak area. Sections follow the same order: **what it is**, **the facts the exam tests**, **syntax**, and **traps**. Every claim was checked against the linked docs.databricks.com page on 2026-10-05. Where a page is vague or a detail was not confirmed in the doc, the guide says so.

#	Topic	Exam domain (DE Associate guide)	One-line takeaway
1	Triggers	Development & Ingestion / Data Processing	availableNow = incremental batch; Once is deprecated
2	ANSI mode	Data Processing & Transformations	ANSI on: bad cast/overflow/div-by-0 throws; use try_*
3	Spark conf	Databricks Intelligence Platform	Compute vs session scope; serverless allows 6 props
4	Dashboard task	Productionizing Data Pipelines	Job task that refreshes a published AI/BI dashboard
5	SET MANAGED	Data Governance & Quality	ALTER TABLE ... SET MANAGED; 14-day UNSET window
6	UC privileges	Data Governance & Quality	SELECT + USE SCHEMA + USE CATALOG to read

NOTE: Domain mapping is approximate. Confirm each objective against the "Sections and Objectives" list in the exam guide PDF.

TOPIC 1

1. Structured Streaming Triggers

When and how often a micro-batch runs

A **trigger** defines *when* a streaming query processes data. It is the main lever for trading off **latency** against **cost**. Choosing the wrong one either wastes compute (polling too often) or leaves data stale.

1.1 The five trigger types

Trigger	Syntax (Python)	Behavior	Status
Unspecified (default)	*no .trigger() call*	Same as processingTime = 0 ms. Next micro-batch starts as soon as the previous one ends. ~3-5 s latency.	Supported. Risk: many cloud storage API calls
processingTime	.trigger(processingTime='10 seconds')	Fixed-interval micro-batches. Avoids checking for new data too often.	Recommended for always-on streams
availableNow	.trigger(availableNow=True)	Processes **all data available at start**, possibly in multiple batches honoring rate limits, then **stops**.	Recommended for all incremental batch
once	.trigger(once=True)	Processes available data then stops.	**Deprecated** since DBR 11.3 LTS
realTime	.trigger(realTime='5 minutes')	Real-time mode: end-to-end latency <1 s (typ. ~300 ms). Argument = micro-batch length.	For operational low-latency use cases
continuous	.trigger(continuous='1 second')	Experimental in OSS Spark.	**Not supported** on Databricks; use real-time mode

EXAM FOCUS: "Use **Trigger.AvailableNow** for all incremental batch processing workloads." If a question describes a job that runs on a schedule (e.g., hourly), processes only new data, and then shuts down the cluster, the answer is **availableNow**, not Once and not processingTime.

1.2 availableNow vs once

Both process what is available and stop. The key difference is **batch sizing**: availableNow respects rate-limit options such as `maxFilesPerTrigger` and `maxBytesPerTrigger`, so a large backlog is split into several manageable micro-batches. Once tries to process everything in one batch, which can cause out-of-memory or very long batches on a large backlog.

■ Incremental batch with Auto Loader + availableNow

```
(spark.readStream
   .format('cloudFiles')
   .option('cloudFiles.format', 'json')
   .option('cloudFiles.maxFilesPerTrigger', 1000)   # batch sizing
   .load('/Volumes/main/raw/landing/orders')
 .writeStream
   .option('checkpointLocation', '/Volumes/main/raw/chk/orders')
   .trigger(availableNow=True)                       # run, drain backlog, stop
   .toTable('main.bronze.orders'))
```

■ Equivalent syntax

```
.trigger(processingTime='30 seconds')   # Python
.trigger(Trigger.ProcessingTime("30 seconds"))   // Scala
.trigger(Trigger.AvailableNow())       // Scala
```

1.3 Source support for availableNow

Source	Minimum Databricks Runtime
File sources (JSON, Parquet, etc.)	9.1 LTS
Delta Lake	10.4 LTS
Apache Kafka	10.4 LTS
Amazon Kinesis	13.1

1.4 Serverless compute

On serverless compute, only **AvailableNow** and **Once** are supported, and AvailableNow is recommended. Interval-based (processingTime) and default triggers are not available there.

WARNING: Trap: "Use serverless compute for a stream that must process every 10 seconds." On serverless you cannot set processingTime. Expect answer options that mix serverless with processingTime to be wrong.

1.5 Cost guidance

- Default trigger checks for data continuously, which can produce many storage LIST/GET calls and unexpected cloud bills.
- Pick the interval from **data arrival rate** and **latency requirement**, not "as fast as possible".
- Latency need of minutes/hours plus intermittent data: schedule a job with availableNow and let the compute shut down.
- Sub-second need (fraud detection): real-time mode, not continuous.

Source: https://docs.databricks.com/aws/en/structured-streaming/triggers

TOPIC 2

2. ANSI SQL Compliance

spark.sql.ansi.enabled and ANSI_MODE

`spark.sql.ansi.enabled` switches Spark SQL from the Hive-compatible dialect to an ANSI-compliant dialect. The main effect: invalid inputs **throw runtime errors instead of silently returning NULL** or wrapped values.

2.1 Defaults

Environment	Default	Setting name
Databricks Runtime 17.0 and above / Apache Spark 4.0	**Enabled**	spark.sql.ansi.enabled
Serverless notebooks and jobs	**true**	spark.sql.ansi.enabled (one of 6 settable props)
Databricks SQL (SQL warehouses)	See ANSI_MODE parameter doc	ANSI_MODE (SET ANSI_MODE = false)
Older Databricks Runtime versions	Disabled	spark.sql.ansi.enabled

NOTE: Databricks SQL uses the parameter name **ANSI_MODE**, not spark.sql.ansi.enabled. The ANSI compliance page points to the ANSI_MODE doc for the DBSQL default, so check that page before the exam.

2.2 Behavior side by side

Expression	ANSI OFF (legacy)	ANSI ON
SELECT 2147483647 + 1	-2147483648 (wraps around)	ERROR: integer overflow
Decimal overflow	NULL	ERROR: arithmetic exception
CAST('a' AS INT)	NULL	ERROR: malformed input
CAST(2147483648L AS INT)	wrapped value	ERROR: overflow
CAST(DATE'2020-01-01' AS INT)	NULL	ERROR: data type mismatch
1 / 0	NULL	ERROR: division by zero
element_at(array, 10) out of range	NULL	ERROR: index out of bounds
to_date('bad')	NULL	ERROR: cannot parse

2.3 The try_ escape hatches

When ANSI is on and you *want* NULL instead of a failure (for example, dirty bronze data), use the **try_** variants. Each behaves exactly like the regular operator but returns NULL instead of throwing.

Function	Replaces	Returns NULL when
try_cast(expr AS type)	CAST	the cast is invalid
try_add(a, b)	+	integral overflow
try_divide(a, b)	/	divisor is 0
try_subtract / try_multiply	- / *	overflow
try_to_timestamp, try_element_at, ...	to_timestamp, element_at	parse fails / index out of range

■ **ANSI on + try_ functions**

```
SET spark.sql.ansi.enabled = true;
```

```
SELECT try_cast('abc' AS INT);      -- NULL (no error)
SELECT try_divide(10, 0);           -- NULL
SELECT CAST('abc' AS INT);          -- fails: CAST_INVALID_INPUT
```

WARNING: try_subtract, try_multiply, try_to_timestamp and try_element_at are standard Databricks SQL functions. The ANSI page itself names only try_cast, try_add and try_divide, so verify the others in the SQL functions reference.

2.4 Store assignment policy (INSERT)

`spark.sql.storeAssignmentPolicy` (default **ANSI**) controls the implicit cast when you insert into a table. Under ANSI, an invalid cast is rejected, so bad values never get written partway. LEGACY restores Spark 2.x behavior (writes NULL or wrong values).

■ Store assignment = ANSI

```
CREATE TABLE test (i INT);
INSERT INTO test VALUES (2147483648L);
-- ERROR [CAST_OVERFLOW_IN_TABLE_INSERT]
```

2.5 Type coercion and reserved keywords

- Precedence (narrow to wide): Byte, Short, Int, Long, Decimal, Float, Double; Date, Timestamp. String can promote to any simple type; NULL to any type.
- Float is skipped when resolving the least common type to avoid precision loss: `typeof(coalesce(1, 1F))` returns **DOUBLE**.
- Reserved keywords (SELECT, FROM, TABLE, ...) become invalid as identifiers only when **both** ansi.enabled and `spark.sql.ansi.enforceReservedKeywords` are true. The latter defaults to **false**.

EXAM FOCUS: Typical question: "A pipeline started failing after moving to a new runtime / serverless with CAST_INVALID_INPUT errors. What is the minimal fix that keeps bad rows as NULL?" Answer: replace CAST with **try_cast**. Disabling ANSI also works but is not best practice.

Source: https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-ansi-compliance

TOPIC 3

3. Spark Configuration Properties

Where to set them, and what serverless allows

Spark properties can be set at two scopes. The scope decides who is affected and for how long.

Scope	How	Affects	Lifetime
Compute (cluster)	Compute UI: Advanced options, Spark tab (key value per line); or spark_conf in the Clusters/Jobs API	All notebooks and jobs on that compute	Until compute config changes
Notebook / session	SET in SQL; spark.conf.set() in Python/Scala	Only the current SparkSession	Until session ends / detach
SQL warehouse	Admin settings (workspace-level data access config); SET for a limited set of aliased parameters	Queries on DBSQL	Workspace / session

3.1 Session-level syntax

■ **SQL**

```
SET spark.sql.ansi.enabled = true;
```

■ **Python**

```
spark.conf.set("spark.sql.ansi.enabled", "true")
spark.conf.get("spark.sql.ansi.enabled")   # read current value
```

■ **Scala**

```
spark.conf.set("spark.sql.ansi.enabled", "true")
```

■ **Databricks SQL warehouse**

```
SET ANSI_MODE = false;          -- DBSQL session override
```

3.2 Serverless: only six properties

Serverless notebooks and jobs let you set only the properties below. Anything else is rejected.

Property	Default	Purpose
spark.databricks.execution.timeout	9000 (notebooks)	Query execution timeout (seconds)
spark.sql.legacy.timeParserPolicy	CORRECTED	Datetime parsing behavior
spark.sql.session.timeZone	Etc/UTC	Session time zone
spark.sql.shuffle.partitions	auto	Default shuffle partitions
spark.sql.ansi.enabled	true	ANSI SQL mode
spark.sql.files.maxPartitionBytes	134217728 (128 MB)	Max bytes per file-read partition

EXAM FOCUS: Memorize at least: **shuffle.partitions**, **session.timeZone**, **ansi.enabled**, **maxPartitionBytes**. A question listing spark.executor.memory or spark.dynamicAllocation.* as something to set on serverless is a distractor.

3.3 Not supported / restricted

Property	Why / alternative
spark.dynamicAllocation.*	Use Databricks autoscaling instead
spark.master, spark.driver.host, spark.driver.port	Restricted on serverless and Lakeflow pipelines

Property	Why / alternative
spark.jars	Not supported on serverless; use serverless JAR tasks
spark.databricks.runtimeoptions.*	Use the compute runtime_options attribute

3.4 Best practices

- Databricks recommends **not** setting most Spark properties. Legacy configs copied from OSS Spark can override newer, optimized defaults (for example, AQE and auto-tuned shuffle partitions).
- Set at the **session** level when only one notebook needs a change; set at the **compute** level when every workload on that compute needs it.
- Never hard-code credentials in Spark config. Reference a secret with `{{secrets/<scope>/<key>}}` in the compute Spark config.

WARNING: The {{secrets/scope/key}} syntax is documented in the secrets docs. It was not re-confirmed on the Spark conf page for this guide, so verify it under docs.databricks.com "Secrets" before relying on it.

Source: https://docs.databricks.com/aws/en/spark/conf

TOPIC 4

4. Dashboard Task in Lakeflow Jobs

Refresh AI/BI dashboards inside a workflow

The **Dashboard** task type refreshes a **published** AI/BI dashboard as one step of a job. Typical pattern: ingest, transform, then refresh the dashboard so it shows the new data immediately, instead of relying on a separate time-based dashboard schedule that may run before the pipeline finishes.

Typical DAG

```
[ingest_bronze] --> [build_silver_gold] --> [refresh_sales_dashboard (Dashboard task)]
```

4.1 Configuration fields

Field	Required?	Notes
Task name / Type = Dashboard	Yes	Select Dashboard from the type dropdown
Dashboard	Yes	Pick a published dashboard you can access
SQL warehouse	Optional	Defaults to the dashboard's warehouse. Must be **serverless or pro** (not classic)
Subscribers	Optional	Users/destinations that get a snapshot email after refresh
Filters	Optional	Pre-set values for filter widgets
Duration threshold, notifications, retries	Optional	Standard job task settings

4.2 Permissions and identity

- Whoever configures the task needs at least **CAN VIEW** on the dashboard, and the dashboard must be published.
- If the dashboard is published **with shared data permissions** (embedded credentials), the refresh runs with the **publisher's** credentials.
- If published **without** shared data permissions, the job's **Run as** identity (defaults to the job owner) runs the refresh.
- Viewers keep their own data permissions. They cannot see results produced under another identity.

4.3 Parameters

Dataset parameters cannot be set directly in the task. Attach the parameter to a filter widget on the dashboard first, then set that filter's value in the task. Job-level parameters are automatically applied to dashboard filters with the matching URL identifier.

EXAM FOCUS: Scenario: "Refresh the dashboard only after the gold tables finish updating, and email stakeholders." Answer: add a **Dashboard task** that depends on the gold task and add **subscribers**. A dashboard schedule cannot wait for pipeline completion.

WARNING: Classic SQL warehouses are not supported for this task. Only serverless or pro.

Source: https://docs.databricks.com/aws/en/jobs/dashboard

TOPIC 5

5. Convert External Tables to Managed

ALTER TABLE ... SET MANAGED

Unity Catalog **managed tables** let Databricks own storage, layout and maintenance (predictive optimization, automatic liquid clustering, etc.), which reduces cost and removes path management. SET MANAGED converts an existing external or foreign Delta table **in place**, keeping its name, history and grants, without a manual CTAS/DEEP CLONE and rename.

5.1 Syntax

■ **Convert, verify, roll back**

```
-- External UC table
ALTER TABLE main.sales.orders SET MANAGED;

-- Foreign table (HMS / Glue federation)
ALTER TABLE hms_cat.sales.orders SET MANAGED MOVE;   -- or COPY

-- Verify
DESCRIBE EXTENDED main.sales.orders;   -- Type: MANAGED

-- Roll back (within 14 days)
ALTER TABLE main.sales.orders UNSET MANAGED;
```

WARNING: MOVE / COPY applies to **foreign** tables only. Adding them to an external-table conversion fails with DELTA_ALTER_TABLE_SET_MANAGED_UNSUPPORTED_COPY_MOVE_SYNTAX.

5.2 Requirements

Requirement	Detail
Compute	Databricks Runtime **17.3 LTS+** or serverless
Format	**Delta Lake** only
Permissions (external)	**Owner** of the table
Permissions (foreign)	OWNER or MANAGE on table + CREATE on the EXTERNAL LOCATION
Foreign source	HMS federation or Glue federation only
Readers / writers	Should be on DBR **15.4 LTS+**. On 14.3 LTS and below, timestamp time travel is restricted after conversion
External clients	Must support reading UC managed tables

5.3 What happens during conversion

Phase	What happens	Downtime
1. Initial copy	Data copied to the managed location while readers and writers keep working	None
2. Switch	Metadata updated to point to the managed location; writes briefly blocked	About 1-2 min up to 1 TB; 1-5 min at 10 TB
After	Type = MANAGED; predictive optimization enabled automatically (unless disabled); original data kept 14 days	-

5.4 Rollback and side effects

- **UNSET MANAGED** works only within **14 days** and needs DBR 17.3 LTS+ or serverless.
- Streaming jobs must be **restarted** after conversion or rollback.
- Foreign tables converted with MOVE need a DROP TABLE after rollback to re-federate.
- Delta Sharing shares must be re-shared with ALTER SHARE.
- Foreign COPY leaves two copies; you are responsible for retiring the source.
- Path-based access (`delta.`/path``) is redirected on DBR 18.1+, at a cost of several hundred ms per operation. Migrate code to `catalog.schema.table` names.

EXAM FOCUS: Key facts to memorize: **SET MANAGED** / **UNSET MANAGED**, **14-day** rollback, **DBR 17.3 LTS+**, **Delta only**, near-zero downtime, predictive optimization turns on. If asked for the "simplest way to move an external Delta table to managed while keeping history", it is SET MANAGED, not CTAS.

Source: https://docs.databricks.com/aws/en/tables/convert-to-managed

TOPIC 6

6. Unity Catalog Privileges

Hierarchy, inheritance, and the 3-privilege rule

6.1 Securable hierarchy

■ **Three-level namespace: catalog.schema.object**

```
METASTORE
 +-- CATALOG                    USE CATALOG
      +-- SCHEMA                USE SCHEMA
           +-- TABLE / VIEW / MV  SELECT, MODIFY, ...
           +-- VOLUME             READ VOLUME, WRITE VOLUME
           +-- FUNCTION / MODEL   EXECUTE
 +-- STORAGE CREDENTIAL, EXTERNAL LOCATION, CONNECTION, SHARE, ...
```

6.2 The rule most often tested

EXAM FOCUS: To read a table a user needs **all three**: **USE CATALOG** on the catalog, **USE SCHEMA** on the schema, and **SELECT** on the table (or inherited from schema/catalog). SELECT alone is not enough.

■ **GRANT / REVOKE / SHOW GRANTS / ownership**

```
GRANT USE CATALOG ON CATALOG main          TO `analysts`;
GRANT USE SCHEMA  ON SCHEMA  main.sales    TO `analysts`;
GRANT SELECT      ON TABLE   main.sales.orders TO `analysts`;

-- Schema-wide read, includes FUTURE tables (inheritance)
GRANT SELECT ON SCHEMA main.sales TO `analysts`;

REVOKE SELECT ON TABLE main.sales.orders FROM `analysts`;
SHOW GRANTS ON TABLE main.sales.orders;
ALTER TABLE main.sales.orders OWNER TO `data_eng`;
```

6.3 Inheritance

A privilege granted on a container flows down to **all current and future** child objects.

- SELECT on a **catalog** = SELECT on every current and future table and view in it.
- MODIFY on a **schema** = MODIFY on every current and future table in the schema.
- CREATE TABLE on a **catalog** = can create tables in any existing or future schema (still needs USE CATALOG / USE SCHEMA).
- Metastores created before 2022-08-25 may need an upgrade to privilege model 1.0 for inheritance.

6.4 Privilege reference

Privilege	Applies to	Allows
USE CATALOG / USE SCHEMA	Catalog / Schema	Required to interact with anything inside. Grants no data access by itself
BROWSE	Catalog (inherits)	See object metadata and request access without usage privileges
SELECT	Table, view, MV, share	Read data
MODIFY	Table	INSERT, UPDATE, DELETE (supersedes fine-grained INSERT/UPDATE/DELETE, which are in Beta)

Privilege	Applies to	Allows
CREATE TABLE / SCHEMA / VOLUME / FUNCTION / MATERIALIZED VIEW	Parent container	Create that object type
EXECUTE	Function, model	Invoke a function or load a model for inference
READ VOLUME / WRITE VOLUME	Volume	Read files / add, modify, delete files
READ FILES / WRITE FILES	External location	Direct path access to cloud storage
CREATE EXTERNAL TABLE	External location	Create external tables at that path
REFRESH	Materialized view	Trigger a manual refresh
APPLY TAG	Most securables	Add/edit tags
MANAGE	Any securable	Manage grants, transfer ownership, rename, drop
ALL PRIVILEGES	Any securable	Everything on object and children, **except** MANAGE, EXTERNAL USE LOCATION/SCHEMA, READ METADATA

6.5 Ownership vs MANAGE vs ALL PRIVILEGES

	Owner	MANAGE	ALL PRIVILEGES
Grant/revoke on object	Yes	Yes	No
Rename / drop / transfer ownership	Yes	Yes	No
Read/write data automatically	Yes	No (can grant self)	Yes
Needs parent USE privileges	Yes	Yes (on parents)	Yes

- ALL PRIVILEGES is evaluated at check time, so newly released privileges are included automatically.
- SHOW GRANTS displays only ALL PRIVILEGES, not each implied privilege. Revoking it removes them all.
- Owners do not automatically receive EXTERNAL USE LOCATION / EXTERNAL USE SCHEMA (to prevent data exfiltration).

WARNING: "Owner can grant" and "MANAGE can grant" are both true; "ALL PRIVILEGES can grant" is false. Don't confuse MANAGE with ALL PRIVILEGES.

Source: https://docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/privileges

QUIZ

7. Practice Questions

20 questions, answers inline

Q1. A nightly job must ingest only new files since the last run, split a large backlog into manageable batches, and then stop. Which trigger?

A. Default (unspecified) B. processingTime='24 hours' C. availableNow=True D. once=True

Answer: C. availableNow processes everything available, respects maxFilesPerTrigger/maxBytesPerTrigger, then stops. Once is deprecated and ignores batch sizing.

Q2. What is the default trigger when none is specified?

A. once B. processingTime of 0 ms C. availableNow D. continuous

Answer: B. Micro-batches run back to back as soon as the previous one completes.

Q3. Which triggers are supported on serverless compute?

A. processingTime only B. AvailableNow and Once C. All triggers D. continuous only

Answer: B. AvailableNow is recommended.

Q4. A fraud-detection stream needs sub-second end-to-end latency on Databricks. What should you use?

A. continuous trigger B. processingTime='0 seconds' C. Real-time mode (realTime trigger) D. availableNow

Answer: C. Continuous processing is not supported on Databricks; real-time mode gives ~300 ms latency.

Q5. With ANSI mode enabled, what does SELECT CAST('abc' AS INT) return?

A. NULL B. 0 C. A runtime error D. -1

Answer: C. Invalid casts throw under ANSI. Use try_cast to get NULL.

Q6. A pipeline must keep running when a string column contains bad numbers, storing NULL for them. ANSI mode stays on. Best fix?

A. SET spark.sql.ansi.enabled = false B. Replace CAST with try_cast C. Wrap in COALESCE D. Use storeAssignmentPolicy=LEGACY

Answer: B. try_cast returns NULL instead of failing and keeps ANSI protections elsewhere.

Q7. With ANSI OFF, SELECT 2147483647 + 1 returns:

A. Error B. NULL C. -2147483648 D. 2147483648

Answer: C. Integer overflow wraps (Java semantics). With ANSI ON it errors.

Q8. What is the default value of spark.sql.storeAssignmentPolicy?

A. LEGACY B. STRICT C. ANSI D. NONE

Answer: C. ANSI: invalid casts during INSERT are rejected.

Q9. Which statement sets ANSI mode only for the current notebook session in Python?

A. spark.conf.set("spark.sql.ansi.enabled","true") B. Add it to the cluster Spark tab C. dbutils.conf.set(...) D. %env ANSI=true

Answer: A. spark.conf.set is session-scoped. The cluster Spark tab affects every workload on that compute.

Q10. Which property can you set on serverless notebooks?

A. spark.executor.memory B. spark.dynamicAllocation.enabled C. spark.sql.shuffle.partitions D. spark.jars

Answer: C. It is one of the six supported properties.

Q11. How do you override ANSI behavior for a session on a SQL warehouse?

A. SET spark.sql.ansi.enabled=false B. SET ANSI_MODE = false C. Edit the warehouse Spark tab D. Not possible

Answer: B. DBSQL exposes aliased parameters like ANSI_MODE.

Q12. A dashboard must refresh only after the gold layer finishes, then email stakeholders. What do you configure?

A. A dashboard schedule every hour B. A Dashboard task downstream of the gold task, with subscribers C. A SQL alert D. A notebook calling the REST API

Answer: B. The Dashboard task runs as part of the job DAG and sends snapshots to subscribers.

Q13. Which SQL warehouse types can a Dashboard task use?

A. Classic only B. Serverless or pro C. Any type D. Only all-purpose clusters

Answer: B.

Q14. A dashboard is published WITHOUT shared data permissions. Whose identity runs the refresh in a Dashboard task?

A. The dashboard publisher B. Each subscriber C. The job's Run as identity D. The workspace admin

Answer: C. With shared data permissions it would be the publisher.

Q15. How do you pass a value to a dashboard dataset parameter from a Dashboard task?

A. Set it directly in the task B. Attach it to a filter widget, then set the filter value in the task C. Not possible D. Via a Spark conf

Answer: B. Dataset parameters cannot be set directly in the task.

Q16. Which command converts an external UC Delta table to managed in place?

A. CONVERT TO DELTA B. ALTER TABLE t SET MANAGED C. CREATE TABLE ... DEEP CLONE D. ALTER TABLE t SET TBLPROPERTIES ('managed'='true')

Answer: B.

Q17. After SET MANAGED, how long can you roll back with UNSET MANAGED?

A. 7 days B. 14 days C. 30 days D. Forever

Answer: B. Databricks keeps the original data for 14 days.

Q18. Minimum compute for running ALTER TABLE ... SET MANAGED?

A. DBR 13.3 LTS B. DBR 15.4 LTS C. DBR 17.3 LTS or serverless D. Any SQL warehouse

Answer: C. Readers/writers should be on 15.4 LTS+.

Q19. A user has SELECT on main.sales.orders but gets a permission error. What is most likely missing?

A. MODIFY B. USE CATALOG on main and/or USE SCHEMA on main.sales C. BROWSE D. EXECUTE

Answer: B. Reading needs USE CATALOG + USE SCHEMA + SELECT.

Q20. Which statement about ALL PRIVILEGES is TRUE?

A. It includes MANAGE B. It lets the grantee grant privileges to others C. It covers current and future child objects but excludes MANAGE D. SHOW GRANTS lists each implied privilege

Answer: C. ALL PRIVILEGES excludes MANAGE, EXTERNAL USE LOCATION/SCHEMA and READ METADATA.

CHEAT

8. Cheat Sheet

Last-look summary before the exam

Topic	Must remember
Triggers	Default = processingTime 0. processingTime('N sec') = fixed interval. **availableNow** = drain backlog in rate-limited batches, stop (use for all incremental batch). once = deprecated (DBR 11.3). continuous = not supported, use **realTime** (<1 s). Serverless: AvailableNow/Once only.
ANSI mode	spark.sql.ansi.enabled; on by default in DBR 17.0+ and serverless. DBSQL name: **ANSI_MODE**. ON: overflow, bad cast, divide by zero = **error**. OFF: NULL / wrap. Escape: **try_cast, try_add, try_divide**. storeAssignmentPolicy default ANSI. coalesce(1,1F) is DOUBLE.
Spark conf	Compute scope (Spark tab) vs session scope (SET / spark.conf.set). Serverless allows 6: execution.timeout, timeParserPolicy, session.timeZone, shuffle.partitions, ansi.enabled, files.maxPartitionBytes. No dynamicAllocation, spark.jars. Prefer defaults.
Dashboard task	Refreshes a **published** AI/BI dashboard in a job. Warehouse: serverless or pro. CAN VIEW needed. Subscribers get snapshots. Identity: publisher (shared data perms) or job Run as. Dataset params only via filter widgets.
SET MANAGED	ALTER TABLE t SET MANAGED (external) / SET MANAGED MOVE|COPY (foreign). Delta only, DBR 17.3 LTS+ or serverless, owner required. Copy + brief switch (~1-5 min). Predictive optimization on. **UNSET MANAGED within 14 days**. Restart streams.
UC privileges	Read = **USE CATALOG + USE SCHEMA + SELECT**. Grants on containers inherit to current and future children. MODIFY = insert/update/delete. BROWSE = see metadata without usage. MANAGE = grants/rename/drop/ownership. ALL PRIVILEGES excludes MANAGE.

Source pages

https://docs.databricks.com/aws/en/structured-streaming/triggers
https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-ansi-compliance
https://docs.databricks.com/aws/en/spark/conf
https://docs.databricks.com/aws/en/jobs/dashboard
https://docs.databricks.com/aws/en/tables/convert-to-managed
https://docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/privileges

NOTE: Items flagged as "verify" in this guide: DBSQL ANSI_MODE default, try_ functions beyond try_cast/try_add/try_divide, and the {{secrets/...}} Spark-config syntax.